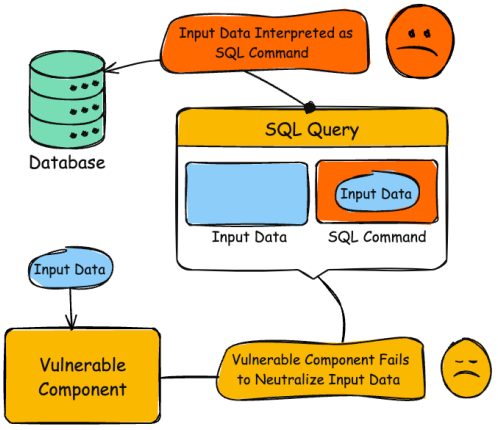
CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Weakness ID: 89
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component. Without sufficient removal or quoting of SQL syntax in user-controllable inputs, the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data.



Alternate Terms

SQL injection	a common attack-oriented phrase
SQLi	a common abbreviation for "SQL injection"

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	**Scope: Confidentiality, Integrity, Availability** Adversaries could execute system commands, typically by changing the SQL statement to redirect output to a file that can then be executed.
Read Application Data	**Scope: Confidentiality** Since SQL databases generally hold sensitive data, loss of confidentiality is a frequent problem with SQL injection vulnerabilities.
Gain Privileges or Assume Identity; Bypass Protection Mechanism	**Scope: Authentication** If poor SQL commands are used to check user names and passwords or perform other kinds of authentication, it may be possible to connect to the product as another user with no previous knowledge of the password.
Bypass Protection Mechanism	**Scope: Access Control** If authorization information is held in a SQL database, it may be possible to change this information through the successful exploitation of a SQL injection vulnerability.
Modify Application Data	**Scope: Integrity** Just as it may be possible to read sensitive information, it is also possible to modify or even delete this information with a SQL injection attack.

Potential Mitigations

Phase(s)	Mitigation

Architecture and Design	**Strategy:** ***Libraries or Frameworks*** Use a vetted library or framework that does not allow this weakness to occur or provides constructs that make this weakness easier to avoid [REF-1482]. For example, consider using persistence layers such as Hibernate or Enterprise Java Beans, which can provide significant protection against SQL injection if used properly.
Architecture and Design	**Strategy:** ***Parameterization*** If available, use structured mechanisms that automatically enforce the separation between data and code. These mechanisms may be able to provide the relevant quoting, encoding, and validation automatically, instead of relying on the developer to provide this capability at every point where output is generated. Process SQL queries using prepared statements, parameterized queries, or stored procedures. These features should accept parameters or variables and support strong typing. Do not dynamically construct and execute query strings within these features using "exec" or similar functionality, since this may re-introduce the possibility of SQL injection. [REF-867]
Architecture and Design; Operation	**Strategy:** ***Environment Hardening*** Run your code using the lowest privileges that are required to accomplish the necessary tasks [REF-76]. If possible, create isolated accounts with limited privileges that are only used for a single task. That way, a successful attack will not immediately give the attacker access to the rest of the software or its environment. For example, database applications rarely need to run as the database administrator, especially in day-to-day operations. Specifically, follow the principle of least privilege when creating user accounts to a SQL database. The database users should only have the minimum privileges necessary to use their account. If the requirements of the system indicate that a user can read and modify their own data, then limit their privileges so they cannot read/write others' data. Use the strictest permissions possible on all database objects, such as execute-only for stored procedures.
Architecture and Design	For any security checks that are performed on the client side, ensure that these checks are duplicated on the server side, in order to avoid CWE-602. Attackers can bypass the client-side checks by modifying values after the checks have been performed, or by changing the client to remove the client-side checks entirely. Then, these modified values would be submitted to the server.
Implementation	**Strategy:** ***Output Encoding*** While it is risky to use dynamically-generated query strings, code, or commands that mix control and data together, sometimes it may be unavoidable. Properly quote arguments and escape any special characters within those arguments. The most conservative approach is to escape or filter all characters that do not pass an extremely strict allowlist (such as everything that is not alphanumeric or white space). If some special characters are still needed, such as white space, wrap each argument in quotes after the escaping/filtering step. Be careful of argument injection (CWE-88). Instead of building a new implementation, such features may be available in the database or programming language. For example, the Oracle DBMS_ASSERT package can check or enforce that parameters have certain properties that make them less vulnerable to SQL injection. For MySQL, the mysql_real_escape_string() API function is available in both C and PHP.
Implementation	**Strategy:** ***Input Validation*** Assume all input is malicious. Use an "accept known good" input validation strategy, i.e., use a list of acceptable inputs that strictly conform to specifications. Reject any input that does not strictly conform to specifications, or transform it into something that does. When performing input validation, consider all potentially relevant properties, including length, type of input, the full range of acceptable values, missing or extra inputs, syntax, consistency across related fields, and conformance to business rules. As an example of business rule logic, "boat" may be syntactically valid because it only contains alphanumeric characters, but it is not valid if the input is only expected to contain colors such as "red" or "blue." Do not rely exclusively on looking for malicious or malformed inputs. This is likely to miss at least one undesirable input, especially if the code's environment changes. This can give attackers enough room to bypass the intended validation. However, denylists can be useful for detecting potential attacks or determining which inputs are so malformed that they should be rejected outright.

When constructing SQL query strings, use stringent allowlists that limit the character set based on the expected value of the parameter in the request. This will indirectly limit the scope of an attack, but this technique is less important than proper output encoding and escaping.

Note that proper output encoding, escaping, and quoting is the most effective solution for preventing SQL injection, although input validation may provide some defense-in-depth. This is because it effectively limits what will appear in output. Input validation will not always prevent SQL injection, especially if you are required to support free-form text fields that could contain arbitrary characters. For example, the name "O'Reilly" would likely pass the validation step, since it is a common last name in the English language. However, it cannot be directly inserted into the database because it contains the "'" apostrophe character, which would need to be escaped or otherwise handled. In this case, stripping the apostrophe might reduce the risk of SQL injection, but it would produce incorrect behavior because the wrong name would be recorded.

When feasible, it may be safest to disallow meta-characters entirely, instead of escaping them. This will provide some defense in depth. After the data is entered into the database, later processes may neglect to escape meta-characters before use, and you may not have control over those processes.

Architecture and Design

Strategy: *Enforcement by Conversion*

When the set of acceptable objects, such as filenames or URLs, is limited or known, create a mapping from a set of fixed input values (such as numeric IDs) to the actual filenames or URLs, and reject all other inputs.

Implementation

Ensure that error messages only contain minimal details that are useful to the intended audience and no one else. The messages need to strike the balance between being too cryptic (which can confuse users) or being too detailed (which may reveal more than intended). The messages should not reveal the methods that were used to determine the error. Attackers can use detailed information to refine or optimize their original attack, thereby increasing their chances of success.

If errors must be captured in some detail, record them in log messages, but consider what could occur if the log messages can be viewed by attackers. Highly sensitive information such as passwords should never be saved to log files.

Avoid inconsistent messaging that might accidentally tip off an attacker about internal state, such as whether a user account exists or not.

In the context of SQL Injection, error messages revealing the structure of a SQL query can help attackers tailor successful attack strings.

Operation

Strategy: *Firewall*

Use an application firewall that can detect attacks against this weakness. It can be beneficial in cases in which the code cannot be fixed (because it is controlled by a third party), as an emergency prevention measure while more comprehensive software assurance measures are applied, or to provide defense in depth [REF-1481.

Effectiveness: Moderate

Note: An application firewall might not cover all possible input vectors. In addition, attack techniques might be available to bypass the protection mechanism, such as using malformed inputs that can still be processed by the component that receives those inputs. Depending on functionality, an application firewall might inadvertently reject or modify legitimate requests. Finally, some manual effort may be required for customization.

Operation; Implementation

Strategy: *Environment Hardening*

When using PHP, configure the application so that it does not use register_globals. During implementation, develop the application so that it does not rely on this feature, but be wary of implementing a register_globals emulation that is subject to weaknesses such as CWE-95, CWE-621, and similar issues.

Relationships

Relevant to the view "Research Concepts" (View-1000)

Nature	Type	ID	Name
ChildOf	C	943	Improper Neutralization of Special Elements in Data Query Logic
ParentOf	V	564	SQL Injection: Hibernate
CanFollow	V	456	Missing Initialization of a Variable

Relevant to the view "Software Development" (View-699)

Nature	Type	ID	Name
MemberOf	C	137	Data Neutralization Issues

Relevant to the view "Weaknesses for Simplified Mapping of Published Vulnerabilities" (View-1003)

Nature	Type	ID	Name
ChildOf	G	74	Improper Neutralization of Special Elements in Output Used by a Downstream Component ('Injection')

Relevant to the view "Architectural Concepts" (View-1008)

Nature	Type	ID	Name
MemberOf	C	1019	Validate Inputs

Relevant to the view "CISQ Quality Measures (2020)" (View-1305)

Nature	Type	ID	Name
ParentOf	V	564	SQL Injection: Hibernate

Relevant to the view "Weaknesses in OWASP Top Ten (2013)" (View-928)

Nature	Type	ID	Name
ParentOf	V	564	SQL Injection: Hibernate

Modes Of Introduction

Phase	Note
Implementation	REALIZATION: This weakness is caused during implementation of an architectural security tactic.
Implementation	This weakness typically appears in data-rich applications that save user inputs in a database.

Applicable Platforms

Languages	Class: Not Language-Specific *(Undetermined Prevalence)* SQL *(Often Prevalent)*
Technologies	Database Server *(Undetermined Prevalence)*

Likelihood Of Exploit

High

Demonstrative Examples

Example 1

In 2008, a large number of web servers were compromised using the same SQL injection attack string. This single string worked against many different programs. The SQL injection was then used to modify the web sites to serve malicious code.

Example 2

The following code dynamically constructs and executes a SQL query that searches for items matching a specified name. The query restricts the items displayed to those where owner matches the user name of the currently-authenticated user.

Example Language: **C#** *(bad code)*

```
...
string userName = ctx.getAuthenticatedUserName();
string query = "SELECT * FROM items WHERE owner = '" + userName + "' AND itemname = '" +
ItemName.Text + "'";
sda = new SqlDataAdapter(query, conn);
DataTable dt = new DataTable();
sda.Fill(dt);
...
```

The query that this code intends to execute follows:

(informative)

```
SELECT * FROM items WHERE owner = <userName> AND itemname = <itemName>;
```

However, because the query is constructed dynamically by concatenating a constant base query string and a user input string, the query only behaves correctly if itemName does not contain a single-quote character. If an attacker with the user name wiley enters the string:

```
(attack code)
name' OR 'a'='a
```

for itemName, then the query becomes the following:

```
(attack code)
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name' OR 'a'='a';
```

The addition of the:

```
(attack code)
OR 'a'='a
```

condition causes the WHERE clause to always evaluate to true, so the query becomes logically equivalent to the much simpler query:

```
(attack code)
SELECT * FROM items;
```

This simplification of the query allows the attacker to bypass the requirement that the query only return items owned by the authenticated user; the query now returns all entries stored in the items table, regardless of their specified owner.

Example 3

This example examines the effects of a different malicious value passed to the query constructed and executed in the previous example.

If an attacker with the user name wiley enters the string:

```
(attack code)
name'; DELETE FROM items; --
```

for itemName, then the query becomes the following two queries:

Example Language: **SQL** (attack code)

```
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name';
DELETE FROM items;
--'
```

Many database servers, including Microsoft(R) SQL Server 2000, allow multiple SQL statements separated by semicolons to be executed at once. While this attack string results in an error on Oracle and other database servers that do not allow the batch-execution of statements separated by semicolons, on databases that do allow batch execution, this type of attack allows the attacker to execute arbitrary commands against the database.

Notice the trailing pair of hyphens (--), which specifies to most database servers that the remainder of the statement is to be treated as a comment and not executed. In this case the comment character serves to remove the trailing single-quote left over from the modified query. On a database where comments are not allowed to be used in this way, the general attack could still be made effective using a trick similar to the one shown in the previous example.

If an attacker enters the string

```
(attack code)
name'; DELETE FROM items; SELECT * FROM items WHERE 'a'='a
```

Then the following three valid statements will be created:

```
(attack code)
```

```
SELECT * FROM items WHERE owner = 'wiley' AND itemname = 'name';
DELETE FROM items;
SELECT * FROM items WHERE 'a'='a';
```

One traditional approach to preventing SQL injection attacks is to handle them as an input validation problem and either accept only characters from an allowlist of safe values or identify and escape a denylist of potentially malicious values. Allowlists can be a very effective means of enforcing strict input validation rules, but parameterized SQL statements require less maintenance and can offer more guarantees with respect to security. As is almost always the case, denylisting is riddled with loopholes that make it ineffective at preventing SQL injection attacks. For example, attackers can:

- Target fields that are not quoted
- Find ways to bypass the need for certain escaped meta-characters
- Use stored procedures to hide the injected meta-characters.

Manually escaping characters in input to SQL queries can help, but it will not make your application secure from SQL injection attacks.

Another solution commonly proposed for dealing with SQL injection attacks is to use stored procedures. Although stored procedures prevent some types of SQL injection attacks, they do not protect against many others. For example, the following PL/SQL procedure is vulnerable to the same SQL injection attack shown in the first example.

Example Language: **SQL** *(bad code)*

```
procedure get_item ( itm_cv IN OUT ItmCurTyp, usr in varchar2, itm in varchar2)
is open itm_cv for
' SELECT * FROM items WHERE ' || 'owner = '|| usr || ' AND itemname = ' || itm || ';
end get_item;
```

Stored procedures typically help prevent SQL injection attacks by limiting the types of statements that can be passed to their parameters. However, there are many ways around the limitations and many interesting statements that can still be passed to stored procedures. Again, stored procedures can prevent some exploits, but they will not make your application secure against SQL injection attacks.

Example 4

MS SQL has a built in function that enables shell command execution. An SQL injection in such a context could be disastrous. For example, a query of the form:

Example Language: **SQL** *(bad code)*

```
SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY='$user_input' ORDER BY PRICE
```

Where $user_input is taken from an untrusted source.

If the user provides the string:

(attack code)

```
'; exec master..xp_cmdshell 'dir' --
```

The query will take the following form:

(attack code)

```
SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY=''; exec master..xp_cmdshell 'dir' --'
ORDER BY PRICE
```

Now, this query can be broken down into:

1. a first SQL query: SELECT ITEM,PRICE FROM PRODUCT WHERE ITEM_CATEGORY='';
2. a second SQL query, which executes the dir command in the shell: exec master..xp_cmdshell 'dir'
3. an MS SQL comment: --' ORDER BY PRICE

As can be seen, the malicious input changes the semantics of the query into a query, a shell command execution and a comment.

Example 5

This code intends to print a message summary given the message ID.

Example Language: **PHP** *(bad code)*

```
$id = $_COOKIE["mid"];
mysql_query("SELECT MessageID, Subject FROM messages WHERE MessageID = '$id'");
```

The programmer may have skipped any input validation on $id under the assumption that attackers cannot modify the cookie. However, this is easy to do with custom client code or even in the web browser.

While $id is wrapped in single quotes in the call to mysql_query(), an attacker could simply change the incoming mid cookie to:

(attack code)

```
1432' or '1' = '1
```

This would produce the resulting query:

(result)

```
SELECT MessageID, Subject FROM messages WHERE MessageID = '1432' or '1' = '1'
```

Not only will this retrieve message number 1432, it will retrieve all other messages.

In this case, the programmer could apply a simple modification to the code to eliminate the SQL injection:

Example Language: **PHP** *(good code)*

```
$id = intval($_COOKIE["mid"]);
mysql_query("SELECT MessageID, Subject FROM messages WHERE MessageID = '$id'");
```

However, if this code is intended to support multiple users with different message boxes, the code might also need an access control check (CWE-285) to ensure that the application user has the permission to see that message.

Example 6

This example attempts to take a last name provided by a user and enter it into a database.

Example Language: **Perl** *(bad code)*

```
$userKey = getUserID();
$name = getUserInput();

# ensure only letters, hyphens and apostrophe are allowed
$name = allowList($name, "^a-zA-z'-$");
$query = "INSERT INTO last_names VALUES('$userKey', '$name')";
```

While the programmer applies an allowlist to the user input, it has shortcomings. First of all, the user is still allowed to provide hyphens, which are used as comment structures in SQL. If a user specifies "--" then the remainder of the statement will be treated as a comment, which may bypass security logic. Furthermore, the allowlist permits the apostrophe, which is also a data / command separator in SQL. If a user supplies a name with an apostrophe, they may be able to alter the structure of the whole statement and even change control flow of the program, possibly accessing or modifying confidential information. In this situation, both the hyphen and apostrophe are legitimate characters for a last name and permitting them is required. Instead, a programmer may want to use a prepared statement or apply an encoding routine to the input to prevent any data / directive misinterpretations.

Selected Observed Examples

Note: this is a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry.

Reference	Description
CVE-2024-6847	SQL injection in AI chatbot via a conversation message
CVE-2025-26794	SQL injection in e-mail agent through SQLite integration
CVE-2023-32530	SQL injection in security product dashboard using crafted certificate fields
CVE-2021-42258	SQL injection in time and billing software, as exploited in the wild per CISA KEV.

CVE-2021-27101	SQL injection in file-transfer system via a crafted Host header, as exploited in the wild per CISA KEV.
CVE-2020-12271	SQL injection in firewall product's admin interface or user portal, as exploited in the wild per CISA KEV.
CVE-2019-3792	An automation system written in Go contains an API that is vulnerable to SQL injection allowing the attacker to read privileged data.
CVE-2004-0366	chain: SQL injection in library intended for database authentication allows SQL injection and authentication bypass.
CVE-2008-2790	SQL injection through an ID that was supposed to be numeric.
CVE-2008-2223	SQL injection through an ID that was supposed to be numeric.
CVE-2007-6602	SQL injection via user name.
CVE-2008-5817	SQL injection via user name or password fields.
CVE-2003-0377	SQL injection in security product, using a crafted group name.
CVE-2008-2380	SQL injection in authentication library.
CVE-2017-11508	SQL injection in vulnerability management and reporting tool, using a crafted password.

Weakness Ordinalities

Ordinality	Description
Primary	*(where the weakness exists independent of other weaknesses)*
Resultant	*(where the weakness is typically related to the presence of some other weaknesses)*

Detection Methods

Method	Details
Automated Static Analysis	This weakness can often be detected using automated static analysis tools. Many modern tools use data flow analysis or constraint-based techniques to minimize the number of false positives. Automated static analysis might not be able to recognize when proper input validation is being performed, leading to false positives - i.e., warnings that do not have any security consequences or do not require any code changes. Automated static analysis might not be able to detect the usage of custom API functions or third-party libraries that indirectly invoke SQL commands, leading to false negatives - especially if the API/library code is not available for analysis. **Note:**This is not a perfect solution, since 100% accuracy and coverage are not feasible.
Automated Dynamic Analysis	This weakness can be detected using dynamic tools and techniques that interact with the software using large test suites with many diverse inputs, such as fuzz testing (fuzzing), robustness testing, and fault injection. The software's operation may slow down, but it should not become unstable, crash, or generate incorrect results. **Effectiveness: Moderate**
Manual Analysis	Manual analysis can be useful for finding this weakness, but it might not achieve desired code coverage within limited time constraints. This becomes difficult for weaknesses that must be considered for all inputs, since the attack surface can be too large.

Automated Static Analysis - Binary or Bytecode	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Bytecode Weakness Analysis - including disassembler + source code weakness analysis • Binary Weakness Analysis - including disassembler + source code weakness analysis **Effectiveness: High**
Dynamic Analysis with Automated Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Database Scanners Cost effective for partial coverage: • Web Application Scanner • Web Services Scanner **Effectiveness: High**
Dynamic Analysis with Manual Results Interpretation	According to SOAR [REF-1479], the following detection techniques may be useful: Cost effective for partial coverage: • Fuzz Tester • Framework-based Fuzzer **Effectiveness: SOAR Partial**
Manual Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Manual Source Code Review (not inspections) Cost effective for partial coverage: • Focused Manual Spotcheck - Focused manual analysis of source **Effectiveness: High**
Automated Static Analysis - Source Code	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Source code Weakness Analyzer • Context-configured Source Code Weakness Analyzer **Effectiveness: High**
Architecture or Design Review	According to SOAR [REF-1479], the following detection techniques may be useful: Highly cost effective: • Formal Methods / Correct-By-Construction Cost effective for partial coverage: • Inspection (IEEE 1028 standard) (can apply to requirements, design, source code, etc.) **Effectiveness: High**

Memberships

Nature	Type	ID	Name
MemberOf	V	635	Weaknesses Originally Used by NVD from 2008 to 2016
MemberOf	C	713	OWASP Top Ten 2007 Category A2 - Injection Flaws
MemberOf	C	722	OWASP Top Ten 2004 Category A1 - Unvalidated Input
MemberOf	C	727	OWASP Top Ten 2004 Category A6 - Injection Flaws
MemberOf	C	751	2009 Top 25 - Insecure Interaction Between Components
MemberOf	C	801	2010 Top 25 - Insecure Interaction Between Components
MemberOf	C	810	OWASP Top Ten 2010 Category A1 - Injection
MemberOf	C	864	2011 Top 25 - Insecure Interaction Between Components
MemberOf	V	884	CWE Cross-section
MemberOf	C	929	OWASP Top Ten 2013 Category A1 - Injection
MemberOf	C	990	SFP Secondary Cluster: Tainted Input to Command
MemberOf	C	1005	7PK - Input Validation and Representation
MemberOf	C	1027	OWASP Top Ten 2017 Category A1 - Injection
MemberOf	C	1131	CISQ Quality Measures (2016) - Security
MemberOf	V	1200	Weaknesses in the 2019 CWE Top 25 Most Dangerous Software Errors
MemberOf	C	1308	CISQ Quality Measures - Security
MemberOf	V	1337	Weaknesses in the 2021 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1340	CISQ Data Protection Measures
MemberOf	C	1347	OWASP Top Ten 2021 Category A03:2021 - Injection
MemberOf	V	1350	Weaknesses in the 2020 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1387	Weaknesses in the 2022 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1409	Comprehensive Categorization: Injection
MemberOf	V	1425	Weaknesses in the 2023 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1430	Weaknesses in the 2024 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	V	1435	Weaknesses in the 2025 CWE Top 25 Most Dangerous Software Weaknesses
MemberOf	C	1440	OWASP Top Ten 2025 Category A05:2025 - Injection

Vulnerability Mapping Notes

Usage	**ALLOWED** *(this CWE ID may be used to map to real-world vulnerabilities)*
Reason	Acceptable-Use
Rationale	This CWE entry is at the Base level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

Notes

Relationship

SQL injection can be resultant from special character mismanagement, MAID, or denylist/allowlist problems. It can be primary to authentication errors.

Taxonomy Mappings

Mapped Taxonomy Name	Node ID	Fit	Mapped Node Name
PLOVER			SQL injection
7 Pernicious Kingdoms			SQL Injection
CLASP			SQL injection
OWASP Top Ten 2007	A2	CWE More Specific	Injection Flaws
OWASP Top Ten 2004	A1	CWE More Specific	Unvalidated Input
OWASP Top Ten 2004	A6	CWE More Specific	Injection Flaws
WASC	19		SQL Injection
Software Fault Patterns	SFP24		Tainted input to command
OMG ASCSM	ASCSM-CWE-89		
SEI CERT Oracle Coding Standard for Java	IDS00-J	Exact	Prevent SQL injection

Related Attack Patterns

CAPEC-ID	Attack Pattern Name
CAPEC-108	Command Line Execution through SQL Injection
CAPEC-109	Object Relational Mapping Injection
CAPEC-110	SQL Injection through SOAP Parameter Tampering
CAPEC-470	Expanding Control over the Operating System from the Database
CAPEC-66	SQL Injection
CAPEC-7	Blind SQL Injection

References

[REF-1460]	rain.forest.puppy. *"NT Web Technology Vulnerabilities".* Phrack Issue 54, Volume 8. 1998-12-25. <https://phrack.org/issues/54/8>. (*URL validated: 2025-07-24*)
[REF-44]	Michael Howard, David LeBlanc and John Viega. *"24 Deadly Sins of Software Security".* "Sin 1: SQL Injection." Page 3. McGraw-Hill. 2010.
[REF-7]	Michael Howard and David LeBlanc. *"Writing Secure Code".* Chapter 12, "Database Input Issues" Page 397. 2nd Edition. Microsoft Press. 2002-12-04. <https://www.microsoftpressstore.com/store/writing-secure-code-9780735617223>.
[REF-867]	OWASP. *"SQL Injection Prevention Cheat Sheet".* <https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html>. (*URL validated: 2025-08-04*)
[REF-868]	Steven Friedl. *"SQL Injection Attacks by Example".* 2007-10-10. <http://www.unixwiz.net/techtips/sql-injection.html>.
[REF-869]	Ferruh Mavituna. *"SQL Injection Cheat Sheet".* 2007-03-15. <https://web.archive.org/web/20080126180244/http://ferruh.mavituna.com/sql-injection-cheatsheet-oku/>. (*URL validated: 2023-04-07*)
[REF-870]	David Litchfield, Chris Anley, John Heasman and Bill Grindlay. *"The Database Hacker's Handbook: Defending Database Servers".* Wiley. 2005-07-14.
[REF-871]	David Litchfield. *"The Oracle Hacker's Handbook: Hacking and Defending Oracle".* Wiley. 2007-01-30.
[REF-872]	Microsoft. *"SQL Injection".* 2008-12. <https://learn.microsoft.com/en-us/previous-versions/sql/sql-server-2008-r2/ms161953(v=sql.105)?redirectedfrom=MSDN>. (*URL validated: 2023-04-07*)

[REF-873]	Microsoft Security Vulnerability Research & Defense. *"SQL Injection Attack".* <https://msrc.microsoft.com/blog/2008/05/sql-injection-attack/>. (*URL validated: 2023-04-07*)
[REF-874]	Michael Howard. *"Giving SQL Injection the Respect it Deserves".* 2008-05-15. <https://learn.microsoft.com/en-us/archive/blogs/michael_howard/giving-sql-injection-the-respect-it-deserves>. (*URL validated: 2023-04-07*)
[REF-875]	Frank Kim. *"Top 25 Series - Rank 2 - SQL Injection".* SANS Software Security Institute. 2010-03-01. <https://www.sans.org/blog/top-25-series-rank-2-sql-injection/>. (*URL validated: 2023-04-07*)
[REF-76]	Sean Barnum and Michael Gegick. *"Least Privilege".* 2005-09-14. <https://web.archive.org/web/20211209014121/https://www.cisa.gov/uscert/bsi/articles/knowledge/principles/least-privilege>. (*URL validated: 2023-04-07*)
[REF-62]	Mark Dowd, John McDonald and Justin Schuh. *"The Art of Software Security Assessment".* Chapter 8, "SQL Queries", Page 431. 1st Edition. Addison Wesley. 2006.
[REF-62]	Mark Dowd, John McDonald and Justin Schuh. *"The Art of Software Security Assessment".* Chapter 17, "SQL Injection", Page 1061. 1st Edition. Addison Wesley. 2006.
[REF-962]	Object Management Group (OMG). *"Automated Source Code Security Measure (ASCSM)".* ASCSM-CWE-89. 2016-01. <http://www.omg.org/spec/ASCSM/1.0/>.
[REF-1447]	Cybersecurity and Infrastructure Security Agency. *"Secure by Design Alert: Eliminating SQL Injection Vulnerabilities in Software".* 2024-03-25. <https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-sql-injection-vulnerabilities-software>. (*URL validated: 2024-07-14*)
[REF-1479]	Gregory Larsen, E. Kenneth Hong Fong, David A. Wheeler and Rama S. Moorthy. *"State-of-the-Art Resources (SOAR) for Software Vulnerability Detection, Test, and Evaluation".* 2014-07. <https://www.ida.org/-/media/feature/publications/s/st/stateoftheart-resources-soar-for-software-vulnerability-detection-test-and-evaluation/p-5061.ashx>. (*URL validated: 2025-09-05*)
[REF-1481]	D3FEND. *"D3FEND: Application Layer Firewall".* <https://d3fend.mitre.org/dao/artifact/d3f:ApplicationLayerFirewall/>. (*URL validated: 2025-09-06*)
[REF-1482]	D3FEND. *"D3FEND: D3-TL Trusted Library".* <https://d3fend.mitre.org/technique/d3f:TrustedLibrary/>. (*URL validated: 2025-09-06*)

Content History

Submissions

Submission Date	Submitter	Organization
2006-07-19 *(CWE Draft 3, 2006-07-19)*	PLOVER	

Contributions

Contribution Date	Contributor	Organization
2024-02-29 *(CWE 4.15, 2024-07-16)*	Abhi Balakrishnan *Provided diagram to improve CWE usability*	

Modifications

Previous Entry Names